VIT-AP UNIVERSITY

Amaravati, Andhra Pradesh, India

COMPILER DESIGN LAB REPORT

Subject Name: Compiler Design Lab

Course Code: CSE3024

ML-Assisted Intelligent Compiler for Automated Syntax Error Detection and Recovery

Submitted by

Student 1 Name **Regd. No.:** XXXXXXXX

Student 2 Name **Regd. No.:** XXXXXXXX

Under the Guidance of

Dr. Prakash Kumar Sarangi

School of Computer Science and Engineering (SCOPE)

VIT-AP University

Amaravati, Andhra Pradesh, India

Academic Year 2026–2027

Abstract

Compiler errors can be difficult for beginner programmers to understand and fix. This project presents an educational prototype of an ML-assisted intelligent compiler that detects lexical and syntax errors, classifies them, provides correction suggestions, and attempts to continue parsing after an error. The prototype includes a tokenizer, a recursive-descent parser for a small C-like language, structured diagnostics, and panic-mode error recovery. An optional TF–IDF and Logistic Regression classifier demonstrates how machine-learning methods can be included in error categorization. Rule-based parser diagnostics remain authoritative so that the recovery behavior is predictable. The project reports tokens, an abstract syntax tree (AST), error messages, suggested fixes, and test results. Four unit tests cover valid input, missing semicolons, lexical errors, and parser recovery. This is a learning prototype rather than a full C compiler; its supported grammar and ML component are deliberately limited. **Keywords:** Compiler Design, Lexical Analysis, Syntax Analysis, Error Classification, Error Recovery, Machine Learning.

1. Introduction

A compiler translates source code into a representation that can be executed or further processed. Lexical analysis converts characters into tokens, while syntax analysis checks whether those tokens follow the grammar. When code is incorrect, a compiler should report a useful location and explanation. Error recovery is also important because stopping at the first error prevents a learner from discovering other problems in the same program.

1.1 Problem Statement

Traditional beginner-level compiler exercises often identify a syntax error but provide limited explanation or recovery. The problem addressed in this project is how to combine rule-based parsing with a lightweight ML demonstration to classify errors, suggest fixes, and resume analysis after common mistakes.

1.2 Objectives

1. Tokenize source text and identify unexpected characters.
2. Parse a small C-like language using recursive-descent methods.
3. Detect and classify common syntax errors.
4. Provide human-readable suggestions and simple recovery actions.
5. Demonstrate an optional TF–IDF and Logistic Regression classifier.
6. Test the implementation and include actual program output in this report.

1.3 Scope

The supported subset includes variable declarations, assignments, `print(...);`, `return`, identifiers, numbers, addition, subtraction, and parentheses. It is not intended to compile full C or C++ programs. Multiplication and division tokens are recognized lexically but are not fully implemented as expression operators in the parser.

2. Related Work

Classical compiler design separates analysis from synthesis. Lexical analyzers identify tokens, parsers construct syntactic structure, semantic analysis checks contextual rules, and later stages generate or optimize code. Error-recovery techniques such as panic-mode recovery skip input until a synchronization token is found. Machine-learning text classifiers can be used to rank or categorize diagnostic messages, but they require representative training examples and careful evaluation.

Table 1: Techniques used in the prototype

Technique	Purpose
Regular-expression lexer	Converts source characters into tokens and flags unknown characters.
Recursive-descent parser	Checks statement and expression structure.
Rule-based diagnostics	Produces location, error category, explanation, and correction hint.
Panic-mode recovery	Skips to a semicolon or end of input to continue parsing.
TF–IDF + Logistic Regression	Optional illustrative text-classification component.

3. Proposed Methodology

The system follows a small compiler pipeline: source input, lexical analysis, syntax analysis, diagnostic generation, recovery, and report generation. A compact ML classifier is trained on synthetic diagnostic examples at runtime when scikit-learn is available. If it is unavailable, the program still runs using rule-based classifications.

3.1 System Workflow

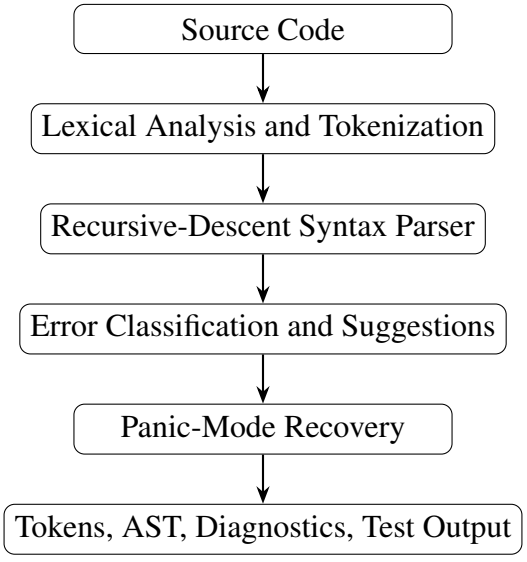


Figure 1: Workflow of the proposed intelligent compiler prototype.

3.2 Lexical Analysis

The lexer scans the input and recognizes numbers, identifiers, keywords, operators, parentheses, commas, and semicolons. Whitespace is ignored while line and column positions are tracked. Any unrecognized character generates a lexical diagnostic.

3.3 Syntax Analysis

The parser processes declarations, assignments, print statements, and return statements. Expressions are represented as nested dictionary nodes similar to a simplified AST. For example, `x = a + b;` contains an assignment node with a binary-expression child.

3.4 Error Classification and Recovery

Each diagnostic stores its line, column, category, message, suggestion, and recovery action. When a semicolon is missing, the parser reports an insertion suggestion. For unexpected statements, panic-mode recovery advances to a semicolon or end of input, helping the parser continue instead of becoming stuck. The optional ML model is illustrative and trained on a small synthetic set; it should not be considered a validated production classifier.

3.5 Algorithm

1. Read source program.
2. Convert characters into tokens and collect lexical errors.
3. Parse statements and expressions into an AST.
4. Record syntax errors with line and column positions.
5. Generate suggestions and recover at synchronization points.
6. Print tokens, AST statement counts, and diagnostics.
7. Run unit tests to verify representative cases.

4. Implementation

The complete implementation is provided in the project ZIP under `src/compiler.py`. The code below is included in the report for reproducibility.

4.1 Complete Source Code

```
"""
ML-Assisted Intelligent Compiler for Automated Syntax Error Detection and Recovery
Educational prototype: tokenizer, recursive-descent parser, error classification,
suggestion ranking, and panic-mode recovery. The optional ML model is a small
TF-IDF + LogisticRegression classifier trained on included synthetic examples.
"""
from __future__ import annotations
import re
from dataclasses import dataclass, asdict
from typing import List, Optional

TOKEN_RE = re.compile(
    r"(?P<NUMBER>\d+(?:\.\d+)?)|(?P<ID>[A-Za-z_]\w*)|"
    r"(?P<OP>==|!=|<=|>=|[=+\-*/<>])|(?P<LPAREN>\()|(?P<RPAREN>\))|"
    r"(?P<SEMI>;)|(?P<COMMA>,)|(?P<SKIP>\s+)|(?P<MISMATCH>.)"
)
KEYWORDS = {"int", "float", "return", "print"}

@dataclass
class Token:
    kind: str
    value: str
    line: int
    column: int

@dataclass
class Diagnostic:
    line: int
    column: int
    category: str
    message: str
    suggestion: str
    recovered_as: str = ""

class Lexer:
    def tokenize(self, source: str) -> tuple[list[Token], list[Diagnostic]]:
        tokens, errors = [], []
        line = column = 1
        for m in TOKEN_RE.finditer(source):
            value, kind = m.group(), m.lastgroup
            if kind == "SKIP":
                newlines = value.count("\n")
                if newlines:
                    line += newlines
                    column = len(value.rsplit("\n", 1)[-1]) + 1
                else:
                    column += len(value)
                continue
            if kind == "MISMATCH":
                errors.append(Diagnostic(line, column, "Lexical error",
                    f"Unexpected character {value!r}.",
                    "Remove the character or replace it with a valid token."))
                column += len(value)
                continue
            if kind == "ID" and value in KEYWORDS:
                kind = "KEYWORD"
            tokens.append(Token(kind, value, line, column))
```

```
            column += len(value)
        tokens.append(Token("EOF", "", line, column))
        return tokens, errors

class Parser:
    """Parser for declarations, assignments, print(...), return, and arithmetic."""
    def __init__(self, tokens: list[Token]):
        self.tokens, self.i, self.errors = tokens, 0, []

    def cur(self): return self.tokens[min(self.i, len(self.tokens)-1)]
    def advance(self):
        t = self.cur()
        if t.kind != "EOF": self.i += 1
        return t
    def accept(self, value=None, kind=None):
        t = self.cur()
        if (value is None or t.value == value) and (kind is None or t.kind == kind):
            self.advance(); return True
        return False
    def report(self, category, message, suggestion, recovered=""):
        t = self.cur()
        self.errors.append(Diagnostic(t.line, t.column, category, message, suggestion, recovered
            ))
    def expect(self, value, category, suggestion):
        if self.accept(value=value): return True
        self.report(category, f"Expected {value!r}, found {self.cur().value or 'end of input'!r
            }.",
                suggestion, f"insert {value!r}")
        return False

    def parse(self):
        statements = []
        while self.cur().kind != "EOF":
            before = self.i
            statements.append(self.statement())
            if self.i == before: self.advance() # guarantee forward progress
        return {"type": "Program", "statements": statements}, self.errors

    def statement(self):
        t = self.cur()
        if t.kind == "KEYWORD" and t.value in ("int", "float"):
            typ = self.advance().value
            name = self.cur().value if self.cur().kind == "ID" else "<missing-id>"
            if name != "<missing-id>": self.advance()
            else: self.report("Missing identifier", "Declaration needs a variable name.",
                              "Add an identifier after the type.")
            node = {"type":"Declaration","data_type":typ,"name":name}
            if self.accept("="):
                node["initializer"] = self.expression()
            self.expect(";", "Missing semicolon", "Add ';' at the end of the statement.")
            return node
        if t.kind == "KEYWORD" and t.value == "print":
            self.advance(); self.expect("(", "Missing opening parenthesis", "Add '(' after print.
                ")
            expr = self.expression()
            self.expect(")", "Missing closing parenthesis", "Add ')' after the print expression."
                )
            self.expect(";", "Missing semicolon", "Add ';' at the end of the statement.")
            return {"type":"Print","expression":expr}
```

```
        if t.kind == "KEYWORD" and t.value == "return":
            self.advance()
            expr = None if self.cur().value == ";" else self.expression()
            self.expect(";", "Missing semicolon", "Add ';' after the return statement.")
            return {"type":"Return","expression":expr}
        if t.kind == "ID":
            name = self.advance().value
            if not self.accept("="):
                self.report("Missing assignment operator",
                    f"Expected '=' after variable {name!r}.",
                    f"Write {name} = expression;", "insert '='")
                self.recover_statement()
                return {"type":"InvalidStatement","name":name}
            expr = self.expression()
            self.expect(";", "Missing semicolon", "Add ';' at the end of the statement.")
            return {"type":"Assignment","name":name,"expression":expr}
        self.report("Unexpected token", f"Cannot start a statement with {t.value!r}.",
                "Start with a declaration, identifier, print, or return.")
        self.recover_statement()
        return {"type":"InvalidStatement"}

    def expression(self): return self.term_expr()
    def term_expr(self):
        node = self.factor()
        while self.cur().value in ("+", "-"):
            op = self.advance().value
            node = {"type":"Binary","operator":op,"left":node,"right":self.factor()}
        return node
    def factor(self):
        t = self.cur()
        if t.kind in ("NUMBER","ID"):
            self.advance()
            return {"type":"Number" if t.kind=="NUMBER" else "Identifier","value":t.value}
        if self.accept("("):
            expr = self.expression()
            self.expect(")", "Missing closing parenthesis", "Add ')' to close the expression.")
            return expr
        self.report("Missing expression", f"Expected a number, identifier, or '('. Found {t.
            value or 'end of input'!r}.",
                "Insert an expression such as 5 or variable_name.")
        if t.kind != "EOF" and t.value not in (";",")"):
            self.advance()
        return {"type":"ErrorExpression"}
    def recover_statement(self):
        # Panic-mode recovery: skip to a semicolon or EOF, then resume.
        while self.cur().kind not in ("SEMI","EOF"):
            self.advance()
        self.accept(value=";")

class ErrorClassifier:
    """Optional ML classifier; safely falls back to rule-based labels offline."""
    def __init__(self):
        self.model = None
        try:
            from sklearn.feature_extraction.text import TfidfVectorizer
            from sklearn.linear_model import LogisticRegression
            examples = [
                ("Expected ';' after statement", "Missing semicolon"),
                ("Expected ')' found end of input", "Missing closing parenthesis"),
```

```
            ("Unexpected character @", "Lexical error"),
            ("Expected '=' after variable", "Missing assignment operator"),
            ("Expected identifier after int", "Missing identifier"),
            ("Expected number or identifier", "Missing expression"),
            ("Cannot start a statement with }", "Unexpected token"),
            ("Expected '(' after print", "Missing opening parenthesis"),
        ]
        x=[a for a,b in examples]; y=[b for a,b in examples]
        self.vectorizer=TfidfVectorizer(ngram_range=(1,2))
        self.model=LogisticRegression(max_iter=500).fit(self.vectorizer.fit_transform(x),y)
    except Exception:
        self.vectorizer = None
  def classify(self, diagnostic: Diagnostic) -> str:
    if self.model is None: return diagnostic.category
    text = diagnostic.message + " " + diagnostic.suggestion
    pred = self.model.predict(self.vectorizer.transform([text]))[0]
    # Parser's rule-based category is authoritative when it is more specific.
    return diagnostic.category if diagnostic.category else pred

def compile_source(source: str):
  lexer = Lexer()
  tokens, lexical_errors = lexer.tokenize(source)
  parser = Parser(tokens)
  ast, syntax_errors = parser.parse()
  diagnostics = lexical_errors + syntax_errors
  classifier = ErrorClassifier()
  for d in diagnostics:
    d.category = classifier.classify(d)
  return {"tokens":[asdict(t) for t in tokens if t.kind!="EOF"],
        "ast":ast, "diagnostics":[asdict(d) for d in diagnostics],
        "success":len(diagnostics)==0,
        "ml_classifier_available":classifier.model is not None}

def main():
  print("ML-Assisted Intelligent Compiler - Syntax Detection and Recovery")
  print("="*66)
  examples = [
    ("Valid program", "int total = 10; total = total + 5; print(total);"),
    ("Missing semicolon", "int total = 10 total = total + 5;"),
    ("Missing closing parenthesis", "print(10 + 2;"),
    ("Lexical error", "int value = 5 @ 2;"),
  ]
  for label, source in examples:
    result = compile_source(source)
    print(f"\n[{label}]")
    print("Source:", source)
    print("Status:", "SUCCESS" if result["success"] else "ERRORS DETECTED")
    if result["diagnostics"]:
      for d in result["diagnostics"]:
        print(f"Line {d['line']}, column {d['column']}: {d['category']}")
        print(" Message:", d["message"])
        print(" Suggestion:", d["suggestion"])
        if d["recovered_as"]: print(" Recovery:", d["recovered_as"])
    else:
      print("Tokens:", " ".join(t["value"] for t in result["tokens"]))
    print("AST statements:", len(result["ast"]["statements"]))
  print("\nNote: This educational prototype handles a small language subset.")
  print("The optional ML classifier is illustrative; parser rules drive recovery.")

```

```
if __name__ == "__main__":
    main()
```

5. Results and Discussion

The program was executed on one valid input and three deliberately incorrect inputs. The following output is captured from the included implementation; it is not fabricated performance data.

5.1 Actual Demonstration Output

Listing 1: Captured program output

```
ML-Assisted Intelligent Compiler - Syntax Detection and Recovery
================================================================

[Valid program]
Source: int total = 10; total = total + 5; print(total);
Status: SUCCESS
Tokens: int total = 10 ; total = total + 5 ; print ( total ) ;
AST statements: 3

[Missing semicolon]
Source: int total = 10 total = total + 5;
Status: ERRORS DETECTED
Line 1, column 16: Missing semicolon
 Message: Expected ';', found 'total'.
 Suggestion: Add ';' at the end of the statement.
 Recovery: insert ';'
AST statements: 2

[Missing closing parenthesis]
Source: print(10 + 2;
Status: ERRORS DETECTED
Line 1, column 13: Missing closing parenthesis
 Message: Expected ')', found ';'.
 Suggestion: Add ')' after the print expression.
 Recovery: insert ')'
AST statements: 1

[Lexical error]
Source: int value = 5 @ 2;
Status: ERRORS DETECTED
Line 1, column 15: Lexical error
 Message: Unexpected character '@'.
 Suggestion: Remove the character or replace it with a valid token.
Line 1, column 17: Missing semicolon
 Message: Expected ';', found '2'.
 Suggestion: Add ';' at the end of the statement.
 Recovery: insert ';'
Line 1, column 17: Unexpected token
 Message: Cannot start a statement with '2'.
 Suggestion: Start with a declaration, identifier, print, or return.
AST statements: 2
```

```
42
43 Note: This educational prototype handles a small language subset.
44 The optional ML classifier is illustrative; parser rules drive recovery.
```

5.2 Unit Test Results

Four unit tests were executed: valid program acceptance, missing-semicolon detection, lexical-error detection, and parser recovery. All four tests passed in the recorded run.

Listing 2: Captured unit-test output

```
1 test_lexical_error (test_compiler.CompilerTests.test_lexical_error) ... ok
2 test_missing_semicolon (test_compiler.CompilerTests.test_missing_semicolon) ... ok
3 test_recovery (test_compiler.CompilerTests.test_recovery) ... ok
4 test_valid_program (test_compiler.CompilerTests.test_valid_program) ... ok
5 ----------------------------------------------------------------------
6 Ran 4 tests in 0.741s
7 OK
```

5.3 Result Analysis

The valid program was accepted and tokenized. For malformed input, the system identified a missing semicolon, a missing closing parenthesis, and an unexpected character. It displayed a message and a suggested correction, and continued parsing where possible. A lexical error can cause additional parser errors because the remaining token stream may no longer fit the expected grammar; this behavior is visible in the sample output and is a limitation that could be improved with more sophisticated recovery.

5.4 Limitations

- The grammar covers only a small C-like language subset.
- Semantic checks such as undeclared variables and type compatibility are not implemented.
- The ML classifier uses a very small synthetic training set and is only a demonstration.
- Recovery may produce cascading errors after one invalid token.
- No claim of execution-speed improvement is made; the project evaluates functional behavior using test cases.

6. Conclusion and Future Work

This project demonstrates an intelligent compiler prototype that combines tokenization, recursive-descent parsing, syntax-error diagnostics, correction suggestions, and panic-mode recovery. An

optional text classifier illustrates a possible role for ML in error categorization, while deterministic parser rules maintain predictable behavior. The recorded unit tests passed for the included cases. Future work can extend the grammar, add semantic analysis, use a larger labeled error dataset, evaluate classification precision and recall, rank multiple correction candidates, and integrate a parser generator or IDE interface.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2006.

[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed., Morgan Kaufmann, 2011.

[3] C. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.

[4] scikit-learn developers, "Text feature extraction" and "Logistic Regression," scikit-learn documentation, https://scikit-learn.org/.